



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECJ3483-01
WEB TECHNOLOGY

Lecturer: DR. NORIZAM BIN KATMON

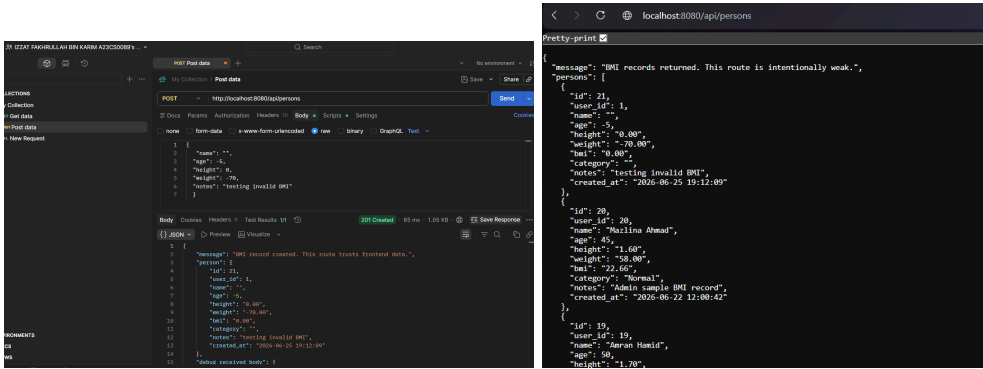
Group Project Lab Exercise

Name	Matric No
IZZAT FAKHRULLAH BIN KARIM	A23CS0089
RAMI YASSEIN ELTAYEB MOHAMED	A23CS0022
AMMAR ABDULRAHMAN ANAAM MUDHSH	A23CS0287
NOUREDIN MAMDOUH M. E. ELNAMLA	A23CS0020

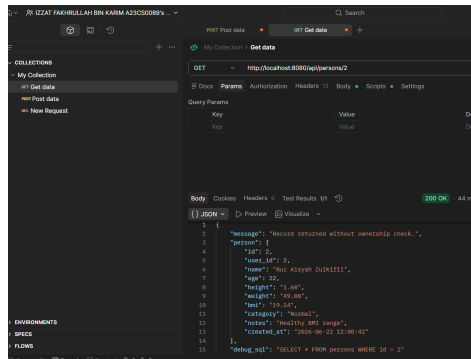
Phase 1: Security Investigation

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness	Why?
1	Add BMI with invalid data	Backend should reject invalid data	Backend accepts the invalid BMI data if sent through Postman/Thunder Client. Frontend validation can be bypassed.	Yes	Invalid data should not be stored because frontend validation can be bypassed.
2	Access another user's BMI record	Backend should deny access	A normal user can request another user's record using API and the backend returns it.	Yes	Users should only access records that belong to them unless they are staff/admin.
3	Access staff route as normal user	Backend should deny access	Normal user can access /api/staff/persons and view all BMI records.	Yes	Normal users should not access staff monitoring features.
4	Access admin route as normal user	Backend should deny access	Normal user can access /api/admin/users and see all users.	Yes	Admin functions must be restricted to admin role only.
5	Submit XSS payload in notes	Payload should display as text	The notes field is rendered using v-html, so payload like can execute in the browser.	Yes	User-generated content should not run as JavaScript in the browser.

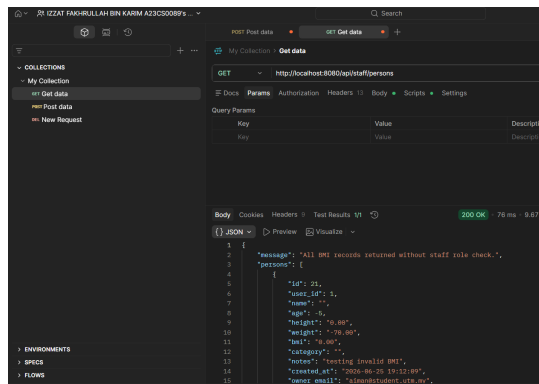
6	Check API response	Password/hash should not be visible	API responses such as /api/admin/users expose password and password_hash	Yes	API should return only necessary data and hide sensitive fields.
7	Try SQL Injection input	Input should be treated as data	Login query uses string concatenation, so SQL injection input may affect the query behavior.	Yes	SQL Injection can happen if input is directly joined into SQL queries.
8	Try to update protected fields	Backend should reject or ignore protected fields	Backend accepts update payloads containing protected fields like user_id, bmi, and category.	Yes	Users should not control fields such as user_id, role, bmi, or category.

No/Title	Screenshots
1-Add BMI with invalid data	

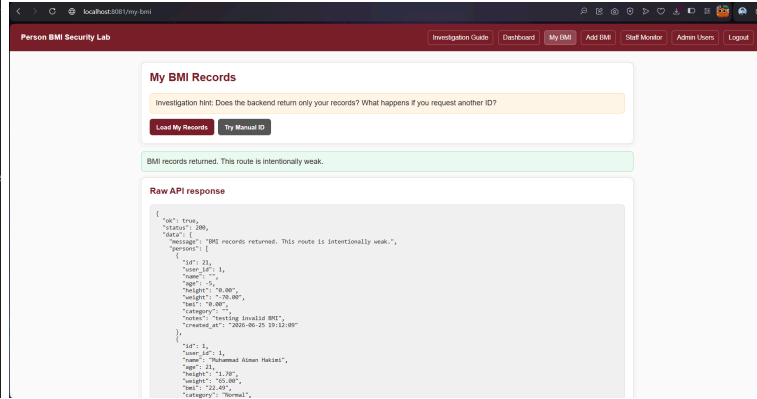
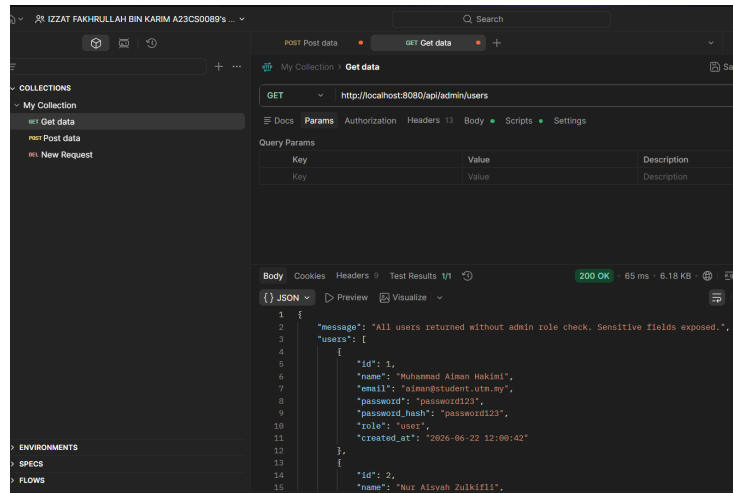
2-Access
another
user's BMI
record



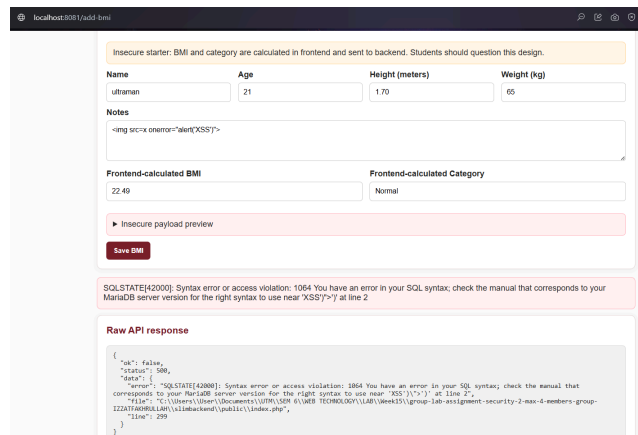
3-Access
staff route as
normal user



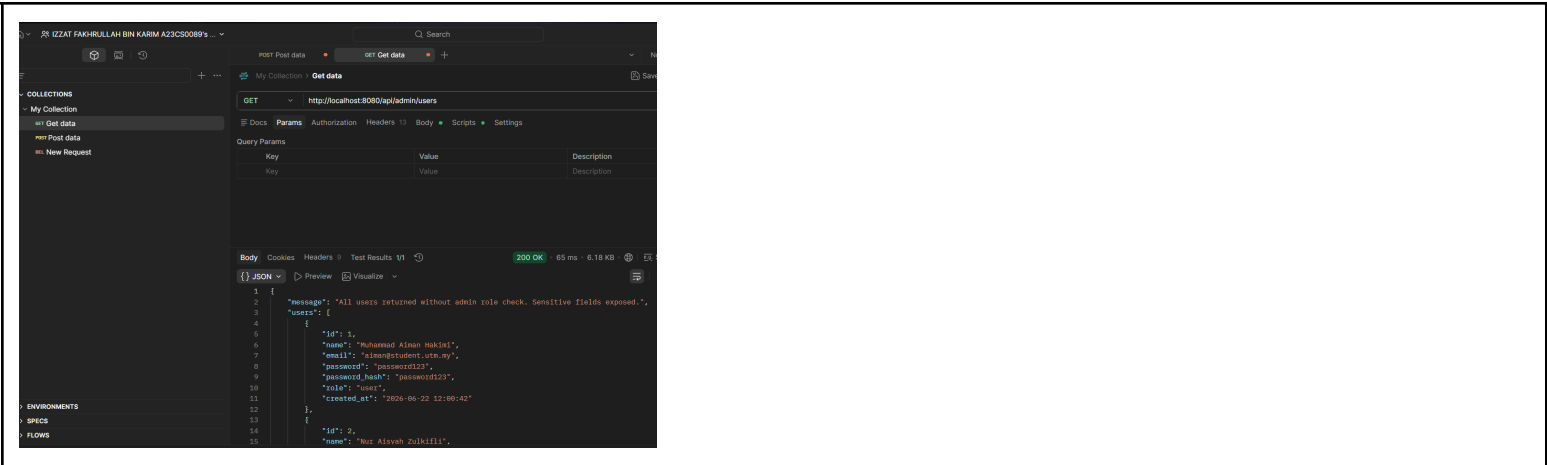
4-Access admin route as normal user



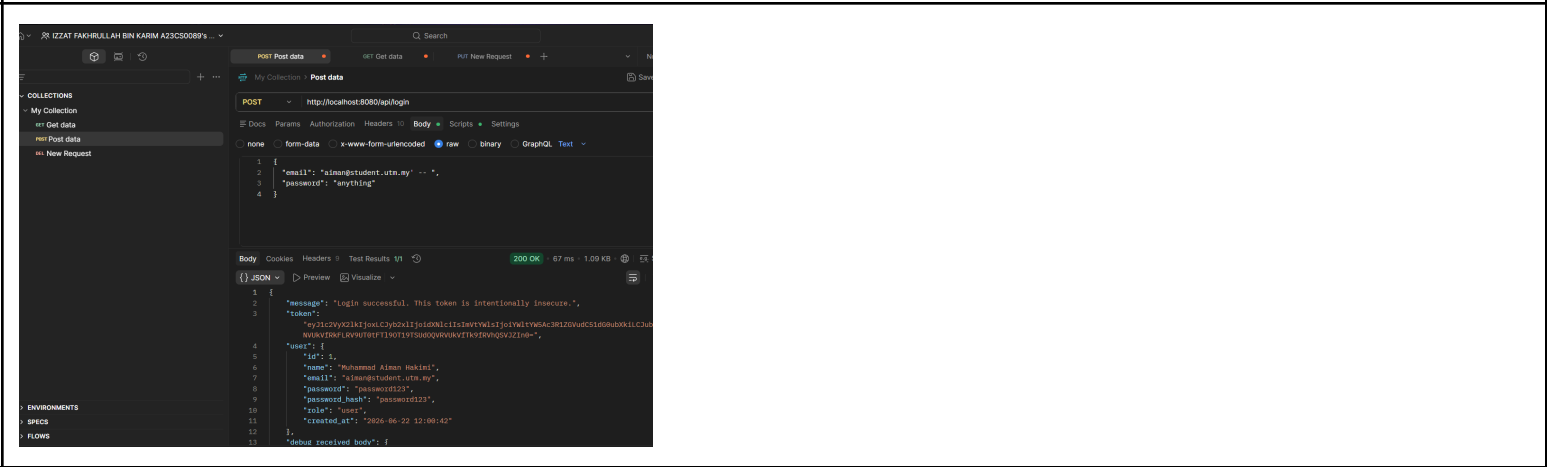
5-XSS payload in notes



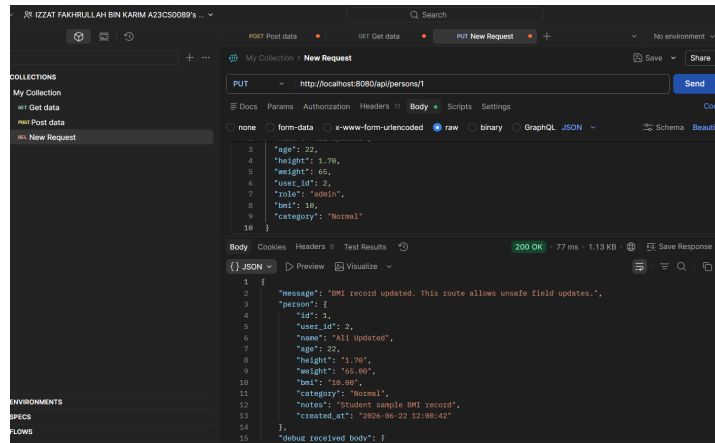
6-API response exposes sensitive data



7-SQL Injection input



8-Update protected fields



The system contains several backend and frontend security weaknesses. The main issue is that the backend trusts user-controlled input and does not fully enforce validation, authentication, authorization, safe API responses, and safe output rendering.

Phase 2: Weakness Classification and Discussion

Weakness Found	Frontend / Backend / API / Database	Security Category	Possible Risk
Invalid BMI data accepted	Backend	Missing backend validation	Invalid BMI data can be stored in the database
User can access another user's BMI record	API / Backend	Broken Object Level Authorization	Privacy violation because users can view records that do not belong to them
Normal user can access staff route	API / Backend	Broken Function Level Authorization	Normal user can view staff-level BMI records
Normal user can access admin route	API / Backend	Broken Function Level Authorization	Unauthorized user may access admin functions
XSS payload executes in notes	Frontend	Cross-Site Scripting	Malicious script can run in the browser
API response exposes password or password_hash	API / Backend	Sensitive Data Exposure	Sensitive user credential information may be exposed
SQL Injection input may affect query	Backend / Database	SQL Injection	Unauthorized access or database query manipulation may happen
User can update protected fields	API / Backend	Mass Assignment	User may change fields such as user_id, role, bmi, or category

Phase 3 : Problem-Solution Mapping

Problem	Root Cause	Where to Fix?	Proposed Solution	How to Retest?
Invalid BMI data accepted	Backend does not validate the input properly	Backend	Validate name, age, height, and weight before saving data	Submit invalid BMI data again using Postman/Thunder Client
User can access another user's BMI record	No ownership check is applied	Backend API	Compare the logged-in user ID from JWT with the record owner ID	Try to access another user's BMI record again
Normal user can access staff route	No role checking for staff route	Backend API	Allow only staff and admin roles to access staff routes	Send request to staff route using normal user token
Normal user can access admin route	No role checking for admin route	Backend API	Allow only admin role to access admin routes	Send request to admin route using normal user token
XSS payload executes in notes	Frontend renders user input as HTML	Frontend	Display user-generated content as text instead of HTML	Submit XSS payload again and check whether alert executes
API response exposes password or password_hash	API returns sensitive user fields	Backend API	Return only necessary fields such as id, name, email, and role	Check API response again and confirm password fields are removed
SQL Injection input may affect query	SQL query uses string concatenation	Backend / Database	Use prepared statements for database queries	Try SQL Injection input again in login or search field
User can update protected fields	Backend accepts fields directly from request body	Backend API	Allow only safe fields such as name, age, height, weight, and notes	Send update request with user_id, role, bmi, or category again